

PRÁCTICA LUCES

apaga.sh

```
#!/bin/bash

# === CONFIGURACIÓN ===
USUARIO="practica"
PASSWORD="cos"
HOST="158.42.181.244"

# === VALIDACIÓN DE ARGUMENTO ===
if [ -z "$1" ]; then
    echo "Uso: $0 <número_de_salida>"
    echo "Debes indicar un número de salida entre 1 y 8"
    exit 1
fi

SALIDA="$1"

# Verificar si es un número entre 1 y 8
if ! [[ "$SALIDA" =~ ^[1-8]$ ]]; then
    echo "Error: '$SALIDA' no es una salida válida."
    echo "Por favor, introduce un número del 1 al 8."
    exit 1
fi

# === ENVÍO DE COMANDOS ===
{
    printf "$USUARIO\n"
    printf "$PASSWORD\n"
    printf "1\n"      # Entrar en el menú de control
    printf "$SALIDA\n" # Seleccionar la salida a encender
    printf "2\n"      # Apagar
    printf "yes\n"     # Confirmar
    printf "\n"        # Retorno
    printf "\033\0334\n" # Salida del menú con doble ESC + 4
} | telnet "$HOST"

echo "Salida $SALIDA encendida correctamente (si no hubo errores de red o autenticación)."
```

apaga_seq.sh

```
#!/bin/bash

echo "Empieza el programa: apaga todas las salidas secuencialmente"

# Dirección IP del dispositivo
HOST="158.42.181.244"

# Número total de salidas
NUM_SALIDAS=8

# Tiempo de retardo entre apagados (en segundos)
RETRASO=1

# Crear la secuencia de comandos que se enviarán por telnet
for ((i=1; i<=NUM_SALIDAS; i++))
do
    echo "Activando salida $i..."

    {
        printf "practica\n"
        printf "cos\n"
        printf "1\n"
        printf "$i\n"
        printf "2\n"
        printf "yes\n"
        printf "\n"
        printf "\033\0334\n"
    } | telnet "$HOST"

    sleep $RETRASO
done

echo "Programa finalizado"
```

apaga_todas.sh

```
#!/bin/bash
echo "empieza el programa apaga_todas.sh\n"

{ printf "practica\ncos\n1\n2\nyes\n\n\033\0334\n"; } | telnet 158.42.181.244

echo "acabado el programa apaga_todas.sh\n"
```

enciende.sh

```
#!/bin/bash

# === CONFIGURACIÓN ===
USUARIO="practica"
PASSWORD="cos"
HOST="158.42.181.244"

# === VALIDACIÓN DE ARGUMENTO ===
if [ -z "$1" ]; then
    echo "Uso: $0 <número_de_salida>"
    echo "Debes indicar un número de salida entre 1 y 8"
    exit 1
fi

SALIDA="$1"

# Verificar si es un número entre 1 y 8
if ! [[ "$SALIDA" =~ ^[1-8]$ ]]; then
    echo "Error: '$SALIDA' no es una salida válida."
    echo "Por favor, introduce un número del 1 al 8."
    exit 1
fi

# === ENVÍO DE COMANDOS ===
{
    printf "$USUARIO\n"
    printf "$PASSWORD\n"
    printf "1\n"      # Entrar en el menú de control
    printf "$SALIDA\n" # Seleccionar la salida a encender
    printf "1\n"      # Encender
    printf "yes\n"     # Confirmar
    printf "\n"        # Retorno
    printf "\033\0334\n" # Salida del menú con doble ESC + 4
} | telnet "$HOST"

echo "Salida $SALIDA encendida correctamente (si no hubo errores de red o autenticación)."
```

enciende_seq.sh

```
#!/bin/bash

echo "Empieza el programa: enciende todas las salidas secuencialmente"

# Dirección IP del dispositivo
HOST="158.42.181.244"
```

```

# Número total de salidas
NUM_SALIDAS=8

# Tiempo de retardo entre apagados (en segundos)
RETRASO=1

# Crear la secuencia de comandos que se enviarán por telnet
for ((i=1; i<=NUM_SALIDAS; i++))
do
    echo "Activando salida $i..."

    {
        printf "practica\n"
        printf "cos\n"
        printf "1\n"
        printf "$i\n"
        printf "1\n"
        printf "yes\n"
        printf "\n"
        printf "\033\0334\n"
    } | telnet "$HOST"

    sleep $RETRASO
done

echo "Programa finalizado"

```

enciende_todas.sh

```

#!/bin/bash
echo "empieza el programa enciende todas\n"

{ printf "practica\ncos\n1\n9\n1\nyes\n\n\033\0334\n"; } | telnet 158.42.181.244

echo "acabado el programa enciende todas\n"

```

generico.sh

```

#!/usr/bin/env bash
#
# pdu: Script genérico para manipular salidas de una PDU por telnet
# Uso:
# pdu -1 <selector_de_salidas> # encender
# pdu -0 <selector_de_salidas> # apagar
# Selectores:
# -a # todas las salidas

```

```
# -n X [-n Y ...] # salida(s) concreta(s)
# -f X           # desde X hasta el final
# -l Y           # desde el principio hasta Y
# -f X -l Y      # rango X..Y
# Opciones:
# -H <host>      # IP/host de la PDU (por defecto: 158.42.181.244)
# -t <segundos>  # retardo entre operaciones (por defecto: 1)
# -h            # ayuda
#
# Notas:
# - NUM_SALIDAS=8 (1..8). Ajusta si tu PDU tiene otro número.
# - La secuencia telnet replica el ejemplo dado; solo varía el estado 1/0.
#
```

```
set -u
```

```
HOST="158.42.181.244" #Maquina por defecto, se puede cambiar entre el rango 241 a 245. Existe un parametro que cambia el host (opcion -H ip/url)
NUM_SALIDAS=8
RETRASO=1
```

```
print_help() {
    cat <<EOF
    Uso: $(basename "$0") <acción> <selección> [opciones]
```

Acciones (una):

```
-1      Encender
-0      Apagar
```

Selección (una o combinación):

```
-a      Todas las salidas (1..${NUM_SALIDAS})
-n X     Solo la salida X (puede repetirse varias veces)
-f X     Desde la salida X hasta ${NUM_SALIDAS}
-l Y     Desde la salida 1 hasta Y
-f X -l Y Rango X..Y
```

Opciones:

```
-H <host> IP/host de la PDU (por defecto: ${HOST})
-t <segundos> Retardo entre operaciones (por defecto: ${RETRASO})
-h        Mostrar esta ayuda
```

Ejemplos:

```
$(basename "$0") -1 -a
$(basename "$0") -0 -n 3
$(basename "$0") -1 -f 2 -l 5
$(basename "$0") -0 -f 6
$(basename "$0") -1 -l 4 -H 192.168.0.10 -t 2
EOF
```

```

}

# Validar entero en rango
in_range() {
    local n="$1"
    [[ "$n" =~ ^[0-9]+$ ]] && (( n >= 1 && n <= NUM_SALIDAS ))
}

# Enviar la secuencia telnet para una salida y acción
# $1: salida (1..NUM_SALIDAS)
# $2: estado ("1" encender, "0" apagar)
send_telnet() {
    local salida="$1"
    local estado="$2"

    echo "→ Operando salida ${salida}: estado=${estado} (${HOST})"

    if(($2 == 0)); then $estado = 2
    fi

    {
        # Credenciales / pasos de menú (según tu ejemplo):
        printf "practica\n" # usuario
        printf "cos\n"      # contraseña
        printf "1\n"        # opción "control" (asumido)
        printf "%s\n" "$salida" # seleccionar salida
        printf "%s\n" "$estado" # 1=ON, 2=OFF
        printf "yes\n"      # confirmar
        printf "\n"
        printf "\033\0334\n" # salir / volver (según ejemplo)
    } | telnet "$HOST"
}

# Parseo de argumentos
if (( $# == 0 )); then
    print_help
    exit 1
fi

ACTION=""
SELECT_ALL=false
declare -a SELECT_NS=()
FROM_VAL=""
TO_VAL=""

# Recorremos args manualmente para permitir flags "sueltos" -1/-0
while (( $# )); do
    case "$1" in

```

```

-h|--help)
    print_help; exit 0
;;
-H)
    shift; [[ $# -gt 0 ]] || { echo "Falta valor para -H"; exit 1; }
    HOST="$1"
    ;;
-t)
    shift; [[ $# -gt 0 ]] || { echo "Falta valor para -t"; exit 1; }
    RETRASO="$1"
    [[ "$RETRASO" =~ ^[0-9]+$ ]] || { echo "Retardo inválido: $RETRASO"; exit 1; }
    ;;
-1|-0)
    if [[ -n "$ACTION" ]]; then
        echo "Error: especifica solo una acción (-1 o -0)"; exit 1
    fi
    ACTION="$1"
    ;;
-a)
    SELECT_ALL=true
    ;;
-n)
    shift; [[ $# -gt 0 ]] || { echo "Falta valor para -n"; exit 1; }
    SELECT_NS+="$1"
    ;;
-f)
    shift; [[ $# -gt 0 ]] || { echo "Falta valor para -f"; exit 1; }
    FROM_VAL="$1"
    ;;
-l)
    shift; [[ $# -gt 0 ]] || { echo "Falta valor para -l"; exit 1; }
    TO_VAL="$1"
    ;;
*)
    echo "Opción desconocida: $1"
    print_help
    exit 1
;;
esac
shift
done

# Validaciones
if [[ -z "$ACTION" ]]; then
    echo "Error: debes indicar una acción (-1 encender, -0 apagar)."
    exit 1
fi

```

```

# Construir lista de salidas
declare -a SALIDAS=()

if $SELECT_ALL; then
  for ((i=1; i<=NUM_SALIDAS; i++)); do SALIDAS+=("$i"); done
fi

for n in "${SELECT_NS[@]"; do
  if ! in_range "$n"; then
    echo "Salida fuera de rango: $n (válidas 1..${NUM_SALIDAS})"; exit 1
  fi
  SALIDAS+=("$n")
done

if [[ -n "$FROM_VAL" && -z "$TO_VAL" ]]; then
  if ! in_range "$FROM_VAL"; then
    echo "Valor -f fuera de rango: $FROM_VAL"; exit 1
  fi
  for ((i=FROM_VAL; i<=NUM_SALIDAS; i++)); do SALIDAS+=("$i"); done
fi

if [[ -z "$FROM_VAL" && -n "$TO_VAL" ]]; then
  if ! in_range "$TO_VAL"; then
    echo "Valor -l fuera de rango: $TO_VAL"; exit 1
  fi
  for ((i=1; i<=TO_VAL; i++)); do SALIDAS+=("$i"); done
fi

if [[ -n "$FROM_VAL" && -n "$TO_VAL" ]]; then
  if ! in_range "$FROM_VAL" || ! in_range "$TO_VAL"; then
    echo "Rango inválido: $FROM_VAL..$TO_VAL (válidas 1..${NUM_SALIDAS})"; exit 1
  fi
  if (( FROM_VAL > TO_VAL )); then
    echo "Rango invertido: -f $FROM_VAL debe ser <= -l $TO_VAL"; exit 1
  fi
  for ((i=FROM_VAL; i<=TO_VAL; i++)); do SALIDAS+=("$i"); done
fi

# Quitar duplicados y ordenar
if (( ${#SALIDAS[@]} == 0 )); then
  echo "Error: no se ha seleccionado ninguna salida. Usa -a, -n, -f y/o -l."
  exit 1
fi

# Ordenar y deduplicar
readarray -t SALIDAS < <(printf "%s\n" "${SALIDAS[@]}" | sort -n | uniq)

# Mapear acción a estado

```

```
ESTADO=""
case "$ACTION" in
  -1) ESTADO="1"; ACCION_TXT="Encender" ;;
  -0) ESTADO="2"; ACCION_TXT="Apagar" ;; # mirar aqui que esto era cero
esac

echo "=== PDU (${HOST}) | ${ACCION_TXT} | Salidas: ${SALIDAS[*]} | Retardo: ${RETRASO}s
=== "

# Bucle principal
for s in "${SALIDAS[@]}"; do
  send_telnet "$s" "$ESTADO"
  sleep "$RETRASO"
done

echo "Hecho."
```